

## Implementace návrhového vzoru Prototype

Představme si situaci, kde máme systém pro správu aut v autopůjčovně. Chceme být schopni vytvářet nové objekty aut, ale chceme se vyhnout opakované inicializaci těchto objektů (např. nastavení vlastností, jako je typ motoru, barva, počet dveří apod.). Místo toho použijeme vzor *Prototype*, který nám umožní klonovat existující objekty.

Vaším úkolem je implementovat systém správy automobilů, kde můžete klonovat prototypy aut. K tomu využijete rozhraní `Prototype<T>`, třídu `Car` a třídu `CarRegistry`, jak je znázorněno na přiloženém **UML diagramu**.

### Požadavky:

#### 1. Rozhraní `Prototype<T>` (viz UML diagram):

- Vytvořte rozhraní `Prototype<T>`, které obsahuje generickou metodu `T clone()`, která vrací klon objektu typu `T`.

#### 2. Třída `Car` (viz UML diagram):

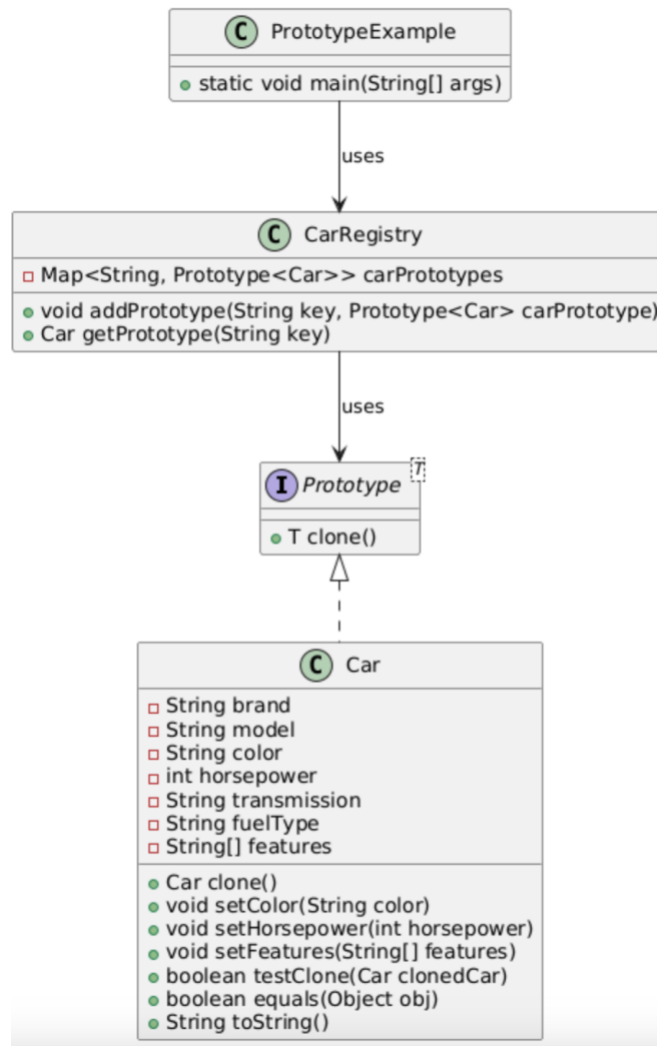
- Implementujte třídu `Car`, která obsahuje atributy jako:
  - `String brand` (značka auta)
  - `String model` (model auta)
  - `Atd` (UML)
- Implementujte metodu `clone()` pro vrácení klonu objektu `Car`.
- Přidejte metodu `testClone(Car clonedCar)`, která otestuje, že klon auta splňuje následující podmínky:
  - Klon není totožný s původním objektem - **`obj.clone() != obj`**
  - Klon je hodnotově ekvivalentní - **`obj.clone().equals(obj)`**
  - Klon má stejnou třídu jako originál - **`obj.clone().getClass() == obj.getClass()`**
  - Překrytá metoda **`equals`** musí porovnávat všechny atributy třídy.

#### 3. Třída `CarRegistry` (viz UML diagram):

- Tato třída bude sloužit jako registr prototypů aut. Uložíte do ní prototypy a budete schopni vytvářet jejich klony.
- Implementujte metody:
  - `void addPrototype(String key, Prototype<Car> carPrototype)`: Přidá auto do registru.
  - `Car getPrototype(String key)`: Vrátil klon auta uloženého pod specifickým klíčem.

#### 4. Hlavní třída `PrototypeExample`:

- Viz obr.



```

public class PrototypeExample {
    public static void main(String[] args) {
        CarRegistry registry = new CarRegistry();

        String[] bmwFeatures = {"Climate Control", "Heated Seats", "Navigation"};
        String[] audiFeatures = {"Cruise Control", "Bluetooth", "Sunroof"};

        Car bmwPrototype = new Car( brand: "BMW", model: "X5", color: "Black", horsepower: 300, transmission: "Automatic", fuelType: "Petrol", bmwFeatures);
        Car audiPrototype = new Car( brand: "Audi", model: "A4", color: "White", horsepower: 220, transmission: "Manual", fuelType: "Diesel", audiFeatures);

        registry.addPrototype( key: "BMW", bmwPrototype);
        registry.addPrototype( key: "Audi", audiPrototype);

        Car car1 = registry.getPrototype( key: "BMW");
        Car car2 = registry.getPrototype( key: "Audi");

        System.out.println("BMW Prototype Clone Test: " + bmwPrototype.testClone(car1));
        System.out.println("Audi Prototype Clone Test: " + audiPrototype.testClone(car2));

        System.out.println(car1);
        System.out.println(car2);

        car1.setColor("Red");
        car2.setHorsepower(250);

        System.out.println("After modification:");
        System.out.println(car1);
        System.out.println(car2);
    }
}
  
```